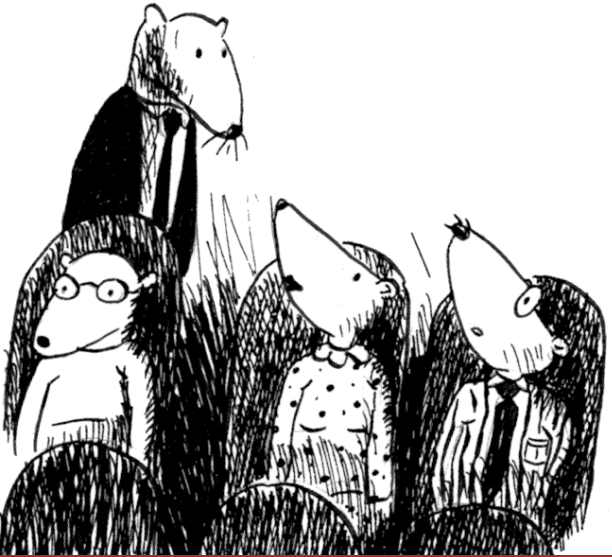


Capítulo 3 — Princípios de Funcionamento de Sistemas Operacionais

Eduardo Camargo de Siqueira

Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
Brasília
Campus São Sebastião



drawn by adit.io

Este material foi criado por Eduardo Camargo de Siqueira e está disponível livremente para uso não comercial, para fins didáticos. Se você usar alguma parte deste material, adicione "copyright duca.pro".

Se você encontrar algum erro ou tiver alguma dúvida, envie um e-mail: eduardo.siqueira@ifb.edu.br. Obrigado por contribuir e espero que você aprenda bastante com este material!

Objetivos do capítulo

Ao final deste capítulo, você deverá ser capaz de:

- compreender o papel do sistema operacional;
- identificar processos de usuário e processos do sistema;
- explicar a divisão de tempo do processador;
- compreender arquivos, diretórios e permissões;
- reconhecer tipos de arquivos que podem se tornar processos.

O que é um sistema operacional?

O sistema operacional é o software responsável por intermediar a relação entre:

- usuários;
- aplicações;
- hardware.

Função central

Gerenciar recursos do computador para que programas possam ser executados de forma controlada, segura e eficiente.

O sistema operacional como camada intermediária

Sem o sistema operacional, cada aplicação precisaria lidar diretamente com detalhes complexos do hardware.

O sistema operacional organiza:

- processador;
- memória;
- arquivos e diretórios;
- dispositivos;
- entrada e saída;
- rede;
- permissões e usuários.

Mais do que uma interface gráfica

Um sistema operacional não é apenas aquilo que aparece na tela.

Ele inclui:

- núcleo do sistema;
- serviços em segundo plano;
- drivers;
- sistema de arquivos;
- mecanismos de segurança;
- controle de processos;
- gerenciamento de memória;
- interfaces gráficas e textuais.

Funções principais

Função	Descrição
Processos	cria, encerra e alterna programas em execução
CPU	divide o tempo do processador
Memória	reserva, protege e libera RAM
Arquivos	organiza dados em arquivos e diretórios
Permissões	controla leitura, escrita e execução
Dispositivos	usa drivers para acessar hardware
Rede	controla comunicação entre computadores
Segurança	protege usuários, arquivos e processos

Gerenciamento de processos

Um processo pode ser entendido como um programa em execução.

O sistema operacional é responsável por:

- criar processos;
- encerrar processos;
- alternar a execução entre processos;
- controlar prioridades;
- permitir comunicação entre processos;
- impedir interferência indevida entre processos.

Programa versus processo

Programa

- arquivo armazenado;
- conjunto de instruções;
- não consome CPU enquanto não executa;
- exemplo: navegador instalado.

Processo

- programa em execução;
- ocupa memória;
- pode usar CPU, arquivos e rede;
- exemplo: navegador aberto.

Ideia importante

Um processo é uma execução concreta de um programa.

Exemplo: criando um processo

Ao executar:

Terminal

```
python3 app.py
```

o sistema operacional cria um processo associado ao interpretador `python3`.

Esse processo:

- pertence ao usuário que executou o comando;
- carrega instruções do arquivo `app.py`;
- usa memória;
- pode acessar arquivos, rede e dispositivos permitidos.

Processos de usuário

Processos de usuário são iniciados direta ou indiretamente por uma pessoa.

Exemplos:

- navegador web;
- editor de texto;
- VS Code;
- terminal;
- tocador de música;
- programa Python;
- servidor local;
- comandos Git;
- contêiner iniciado pelo usuário.

Processos do sistema operacional

Processos do sistema mantêm o computador funcionando.

Eles podem cuidar de:

- inicialização;
- login;
- rede;
- áudio;
- energia;
- impressão;
- atualizações;
- interface gráfica;
- logs;
- dispositivos.

Usuário versus sistema

Aspecto	Usuário	Sistema
Origem	usuário ou aplicação	sistema operacional
Finalidade	tarefa do usuário	manter o sistema ativo
Exemplos	navegador, terminal	rede, login, impressão
Permissões	geralmente limitadas	podem ser elevadas
Visibilidade	muitas vezes visíveis	geralmente em segundo plano
Encerrar	menor risco	pode afetar o sistema

Atenção

Nem todo processo em segundo plano é processo do sistema.

Estados de um processo

Durante sua existência, um processo pode passar por diferentes estados.

- novo;
- pronto;
- em execução;
- bloqueado ou em espera;
- finalizado.

Ideia central

Um processo não fica necessariamente usando CPU o tempo todo.

Estados: visão geral

Estado	Significado
Novo	processo está sendo criado
Pronto	pode executar, mas aguarda CPU
Execução	está usando o processador
Bloqueado	aguarda evento, entrada, disco ou rede
Finalizado	terminou ou foi encerrado

Exemplo

Um programa aguardando `input()` pode estar aberto, mas bloqueado esperando entrada do usuário.

Transições entre estados

Uma sequência simplificada:

1. o processo é criado;
2. passa para pronto;
3. o escalonador escolhe o processo;
4. entra em execução;
5. pode ficar bloqueado aguardando evento;
6. volta para pronto quando o evento ocorre;
7. termina e é finalizado.

Para interpretar desempenho

Nem toda lentidão é causada por falta de processador.

Divisão de tempo do processador

O processador é um recurso limitado e disputado por vários processos.

O sistema operacional precisa decidir:

- qual processo usará a CPU;
- por quanto tempo;
- em qual núcleo;
- com qual prioridade;
- quando interromper ou retomar a execução.

A ilusão de simultaneidade

Para o usuário, vários programas parecem funcionar ao mesmo tempo.

Na prática, o sistema operacional:

- alterna rapidamente entre processos;
- distribui tarefas entre núcleos;
- mantém processos prontos, bloqueados ou em execução;
- cria sensação de resposta contínua.

Analogia

Como um professor que atende vários estudantes alternando rapidamente entre eles.

Escalonador

O escalonador é o componente responsável por escolher qual processo usará a CPU.

Ele pode considerar:

- prioridade;
- tempo de espera;
- tipo de tarefa;
- necessidade de resposta rápida;
- uso anterior da CPU;
- estado do processo;
- quantidade de núcleos disponíveis.

Quantum de tempo

Em muitos sistemas, cada processo recebe uma pequena fatia de tempo para usar a CPU.

Essa fatia é chamada de:

quantum ou *time slice*

Quando o quantum termina:

- o processo pode ser interrompido;
- seu estado é preservado;
- outro processo passa a usar a CPU.

Troca de contexto

A troca de contexto ocorre quando o sistema interrompe um processo e passa a executar outro.

O sistema salva informações como:

- registradores;
- posição atual da execução;
- estado do processo;
- informações de memória;
- dados de controle do sistema.

Custo

Trocas de contexto são necessárias, mas muitas trocas podem prejudicar o desempenho.

Concorrência e paralelismo

Concorrência

- várias tarefas progridem no mesmo período;
- não precisam executar exatamente ao mesmo tempo;
- depende de alternância.

Paralelismo

- várias tarefas executam literalmente ao mesmo tempo;
- depende de múltiplos núcleos ou processadores;
- permite execução simultânea real.

Processos limitados por CPU e por E/S

Limitados por CPU

- cálculos intensos;
- compactação;
- criptografia;
- renderização;
- simulações;
- treinamento de modelos.

Limitados por E/S

- aguardam disco;
- aguardam rede;
- aguardam teclado;
- aguardam banco de dados;
- aguardam resposta externa.

Threads

Uma thread é uma linha de execução dentro de um processo.

Um processo pode ter várias threads para realizar tarefas diferentes.

Exemplo

Um navegador pode usar threads para:

- interface;
- rede;
- renderização;
- execução de scripts.

O escalonador também pode alternar entre threads.

Observando processos

Em Linux:

Comandos

```
ps aux
top
htop
pstree
systemctl status
```

Observando processos

Em Windows:

Comandos e ferramentas

```
tasklist
taskmgr
services.msc
netstat -ano
```

Experimento: CPU intensa

Programa que consome CPU continuamente:

cpu.py

```
while True:
pass
```

Observe com:

Terminal

```
top
htop
```

Experimento: espera por entrada

Programa que aguarda entrada do usuário:

entrada.py

```
nome = input("Digite seu nome: ")
print("Olá,", nome)
```

Enquanto aguarda a digitação, o processo não precisa consumir CPU de forma intensa.

Reflexão

Processo aberto não significa uso contínuo do processador.

Arquivos e diretórios

Uma das funções mais visíveis do sistema operacional é organizar informações em arquivos e diretórios.

Usamos o sistema de arquivos ao:

- salvar documentos;
- baixar imagens;
- instalar programas;
- criar projetos;
- registrar configurações;
- versionar código.

Abstração

Trabalhamos com nomes e caminhos, não com setores físicos do disco.

O que é um arquivo?

Um arquivo é uma unidade de armazenamento de dados.

Pode conter:

- texto;
- imagem;
- áudio;
- vídeo;
- código-fonte;
- dados estruturados;
- configurações;
- instruções executáveis.

Visão do sistema

Um arquivo é uma sequência de bytes cujo significado depende do formato.

Metadados de arquivos

Além do conteúdo, arquivos possuem informações associadas.

Exemplos de metadados:

- nome;
- tamanho;
- extensão;
- tipo;
- data de criação;
- data de modificação;
- permissões;
- proprietário;
- localização.

Nome, extensão, tipo e formato

Extensão

- parte do nome;
- vem após o ponto;
- sugere como abrir o arquivo;
- exemplos: `.txt`, `.py`, `.pdf`.

Formato

- organização interna dos bytes;
- define como interpretar os dados;
- não muda apenas renomeando o arquivo.

Atenção

Renomear `foto.jpg` para `foto.txt` não transforma a imagem em texto.

Extensões comuns

Extensão	Uso comum
`.txt`	texto simples
`.pdf`	documento PDF
`.jpg`, `.png`	imagens
`.csv`	dados tabulares
`.json`	dados estruturados
`.html`, `.css`, `.js`	web
`.py`	código Python
`.java`	código Java
`.sh`	script shell
`.exe`	executável Windows

Diretórios

Um diretório, ou pasta, organiza arquivos e outros diretórios.

Ele permite:

- agrupar informações relacionadas;
- separar projetos;
- criar hierarquias;
- localizar arquivos;
- organizar código, documentação e configurações.

Ideia

Diretórios evitam que todos os arquivos fiquem misturados no mesmo lugar.

Organização em árvore

Exemplo de organização de uma disciplina:

Estrutura

```
sistemas-computacionais/
 aulas/
 exercicios/
 projetos/
 leituras/
```

Raiz

Em Linux e macOS: `/`
Em Windows: unidades como `C:\` e `D:\`

Caminhos absolutos e relativos

Caminho absoluto

- começa na raiz;
- indica localização completa;
- independe do diretório atual.

/home/aluno/projetos/app.py

Caminho relativo

- depende do diretório atual;
- usa referências como `.` e `..`;
- comum em projetos.

../docs/anotacoes.md

Símbolos importantes em caminhos

Símbolo	Significado
.	diretório atual
..	diretório pai
~	diretório pessoal do usuário
/	separador em Linux e macOS
\	separador em Windows

Erro comum

Muitos erros em programação ocorrem porque o arquivo é referenciado a partir do diretório errado.

Recursão de diretórios

Diretórios podem conter outros diretórios, criando uma estrutura recursiva.

Exemplo

```
projeto/
 src/
    controllers/
    models/
    views/
 tests/
    unitarios/
    integracao/
 docs/
 imagens/
```

Comandos recursivos

Comandos recursivos percorrem subdiretórios.

Exemplos

```
ls -R
find . -name "*.py"
du -sh *
rm -r pasta/
```

Cuidado

`rm -r` pode apagar uma pasta inteira e todos os seus conteúdos.

Arquivos ocultos

Em Linux e macOS, arquivos ocultos geralmente começam com ponto.

Exemplos

```
.bashrc
.gitignore
.config/
.ssh/
.git/
```

Para listar:

Terminal

```
ls -a
```

Permissões de arquivos e diretórios

O sistema operacional controla quem pode:

- ler;
- modificar;
- executar;
- acessar diretórios;
- alterar recursos protegidos.

Importância

Permissões são essenciais em servidores, laboratórios, ambientes multiusuário e sistemas compartilhados.

Permissões básicas em Linux

Permissão	Símbolo	Significado
Leitura	`r`	ler arquivo ou listar diretório
Escrita	`w`	modificar arquivo ou diretório
Execução	`x`	executar arquivo ou acessar diretório

$$-rwxr-xr--$$

- dono;
- grupo;
- demais usuários.

Permissão de execução

Nem todo arquivo pode ser executado como programa.

Para tornar um script executável:

Terminal

```
chmod +x backup.sh
./backup.sh
```

Atenção

Dar permissão de execução significa permitir que o arquivo seja tratado como programa ou script.

Arquivos em projetos de software

Uma estrutura simples de projeto Flask:

Projeto

```
meu_app/
 app.py
 requirements.txt
 README.md
 static/
    style.css
 templates/
 index.html
```

Arquivos de configuração

Sistemas e aplicações usam arquivos para guardar parâmetros.

Exemplos:

- `.env`;
- `config.json`;
- `settings.py`;
- `docker-compose.yml`;
- `.gitignore`.

Segurança

Arquivos com senhas, tokens ou chaves de API não devem ser publicados em repositórios públicos.

Operações comuns

Comando	Função
`pwd`	mostra diretório atual
`ls`	lista arquivos
`cd pasta`	entra em diretório
`cd ..`	volta ao diretório pai
`mkdir pasta`	cria diretório
`touch arquivo.txt`	cria arquivo vazio
`cp origem destino`	copia
`mv origem destino`	move ou renomeia
`rm arquivo`	remove arquivo
`find`	procura arquivos

Tipos de arquivos que podem se tornar processos

Alguns arquivos podem dar origem a processos quando executados.

Exemplos:

- executáveis do sistema;
- scripts shell;
- programas Python;
- programas JavaScript em Node.js;
- programas Java compilados;
- binários compilados em C ou C++;
- serviços iniciados pelo sistema.

Ideia

Um arquivo armazenado no disco pode se tornar processo quando carregado e executado pelo sistema operacional.

Relação com desenvolvimento de software

Compreender processos, arquivos e permissões ajuda a resolver problemas como:

- aplicação não inicia;
- porta já está em uso;
- script sem permissão de execução;
- arquivo não encontrado;
- caminho relativo incorreto;
- dependência ausente;
- processo consumindo CPU ou memória;
- configuração exposta indevidamente.

Atividade prática: processos

Execute um servidor local:

Terminal

```
python3 -m http.server 8000
```

Depois, observe:

- o processo aparece na lista?
- ele consome CPU continuamente?
- o que ocorre ao acessar `http://localhost:8000`?
- o que acontece ao encerrar com `Ctrl+C`?
- que estados o processo pode ter assumido?

Atividade prática: arquivos

Crie uma estrutura de projeto:

Terminal

```
mkdir sistema-arquivos
cd sistema-arquivos

mkdir src docs tests
touch README.md
touch src/app.py
touch docs/anotacoes.md
touch tests/test_app.py

ls -R
```

Perguntas para reflexão

- Por que não basta dizer que processo é apenas um programa aberto?
- Como o sistema operacional cria a sensação de simultaneidade?
- O que poderia acontecer se um processo ocupasse a CPU para sempre?
- Por que a memória de um processo deve ser protegida?
- Como permissões ajudam a evitar danos ao sistema?

Mais perguntas para reflexão

- Por que encerrar um processo do sistema pode ser perigoso?
- Como identificar se uma porta já está sendo usada?
- Por que caminhos relativos podem causar erros?
- Por que arquivos de configuração devem ser protegidos?
- Como processos, memória, arquivos e permissões aparecem juntos quando executamos uma aplicação pelo terminal?

Síntese do capítulo

- O sistema operacional intermedia hardware, aplicações e usuários.
- Processos representam programas em execução.
- O escalonador divide o tempo da CPU.
- Arquivos são sequências de bytes organizadas em diretórios.
- Permissões controlam leitura, escrita e execução.
- Compreender esses conceitos ajuda a diagnosticar problemas reais.

FIM

- Dúvidas!?
- http://duca.pro.br/ — eduardo.siqueira@ifb.edu.br
- Ilustrações: copyright Manning Publications, drawn by adit.io
- Obrigado pela atenção!